

Banco de Dados

Aula 5 - CASE e JOIN



MySQL®

CASE

Em alguns casos precisamos substituir o resultado de uma query por palavras específicas e baseadas em condições. Por exemplo, considere um banco de dados de condições ambientais onde queremos apenas apresentar a palavra CALOR como um título para temperaturas acima de 30°C. Para estes casos usamos o CASE, de uma maneira semelhante a que utilizamos em outras linguagens de programação. Dentro do CASE declaramos as condições em que os resultados devem ser alterados. Vejamos um exemplo em nosso banco de escola.

```
SELECT
    nome AS Nome_Professor,
    salario AS Salario,
    -- A Cláusula CASE avalia o valor do salário para definir uma Categoria Salarial
    CASE
        WHEN salario ≥ 9000.00 THEN 'Sênior (Salário Alto)'
        WHEN salario ≥ 7000.00 THEN 'Pleno (Salário Médio)'
        ELSE 'Júnior (Salário Inicial)'
    END AS Categoria_Salarial
FROM
    professores
ORDER BY
    salario DESC;
```

JOIN

Em alguns casos precisamos unir os dados de uma ou mais tabelas para facilitar buscar e relacionar as informações. Para que possamos relacionar os dados de ambas as tabelas é necessário que exista uma coluna com dados comuns entre ambas as tabelas.

Aqui vamos considerar a tabelas `estudantes` dos nossos exemplos como sendo a tabela `A` . A tabela `notas` será a tabela `B` .

```
SELECT * FROM estudantes AS A;  
SELECT * FROM notas AS B;
```

Este exemplo apenas retorna todas as colunas de cada tabela, como já vimos em outros exemplos. Mas o importante aqui é como estamos criando um alias ou "apelido" para cada tabela utilizando o `AS` . Esse alias é importante para facilitar a escrever queries mais curtas, além de ser uma prática comum na construção de queries SQL.

JOIN

Agora devemos indicar ao banco quais colunas queremos comparar para associar informações nas duas tabelas.

Lembre que cada estudante possuem um identificador único na nossa tabela de `estudantes` localizado na coluna `id`. Esse mesmo valor está presente na tabela `notas`, identificado pela coluna `estudante_id`. Desta forma, podemos utilizar essa informação para localizar as notas pertencentes a cada estudante na tabela de notas utilizando o JOIN.

```
SELECT *  
FROM  
    estudantes AS A  
JOIN  
    notas AS B ON A.id = B.estudante_id;
```

Tendo em vista que estamos fazendo um JOIN, ou seja, juntando as informações das duas tabelas, o resultado dessa querie são todas as colunas de ambas as tabelas (pois estamos utilizando o `*`), mas associando cada aluno com suas respectivas notas.

Este exemplo fica um tanto poluído porque apresenta muitas colunas, porém ajuda a identificar que os campos `id` e `estudante_id` são iguais.

JOIN

Vamos agora organizar um pouco melhor as colunas para ter um resultado mais interessante associando as informações das duas tabelas.

```
SELECT
  A.nome AS Nome,
  B.nota AS Nota
FROM
  estudantes AS A
JOIN
  notas AS B ON A.id = B.estudante_id;
```

Agora temos apenas duas colunas, uma com o nome do(a) aluno(a) e sua respectiva nota. Porém, agora parece que temos informação de menos pois não conseguimos identificar a qual disciplina pertence aquela nota. Para resolver este problema podemos fazer mais um JOIN com outra tabela e assim associar as informações das tabelas `estudantes`, `notas` e `disciplinas`.

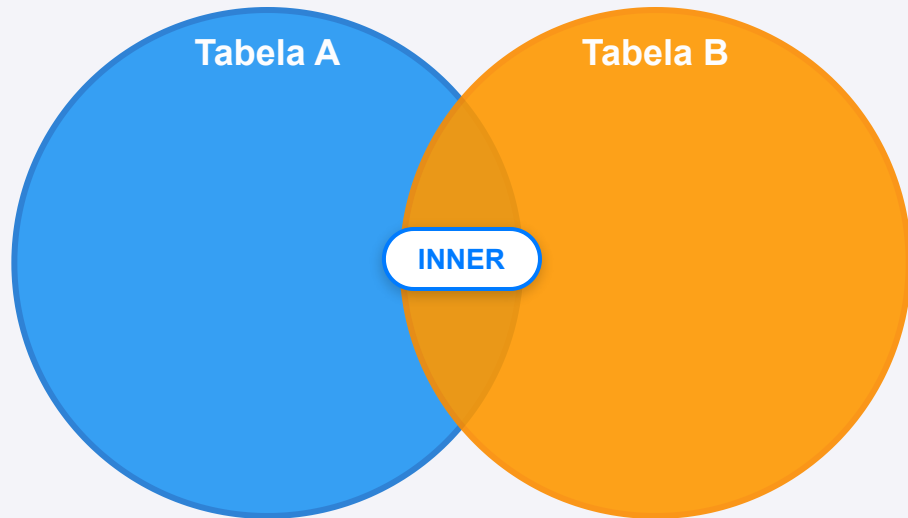
A seguir vamos ver os tipos de JOIN mais utilizados.

O INNER JOIN, ou apenas JOIN, retorna somente os registros que possuem **correspondência em AMBAS as tabelas**.

Se um registro existe apenas na Tabela A ou apenas na Tabela B, ele **não será incluído** no resultado.

```
SELECT *  
FROM  
    estudantes AS A  
JOIN  
    notas AS B ON A.id = B.estudante_id;
```

INNER JOIN

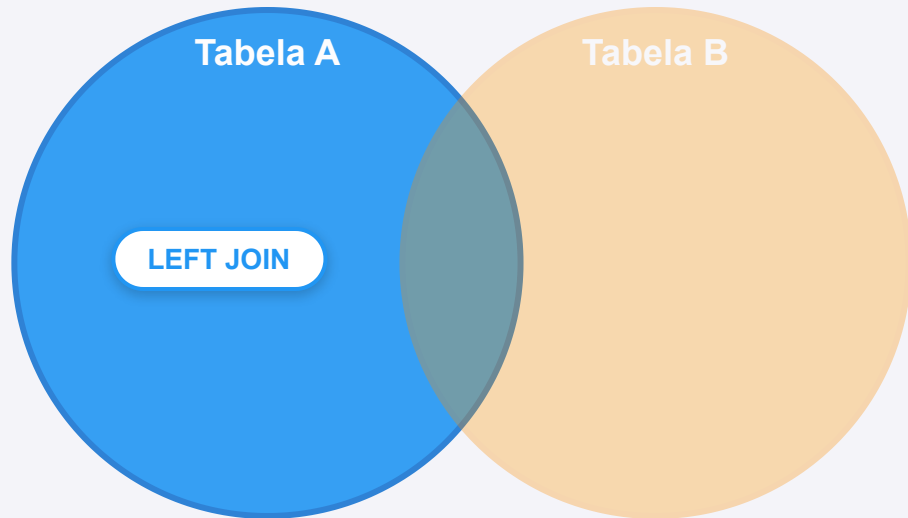


O LEFT JOIN retorna **TODOS os registros da Tabela A (esquerda)**, independentemente de haver correspondência na Tabela B.

Quando há correspondência na Tabela B, os dados são incluídos. Quando não há, os campos da Tabela B aparecem como **NULL**.

```
SELECT *  
FROM  
    estudantes AS A  
LEFT JOIN  
    notas AS B ON A.id = B.estudante_id;
```

LEFT JOIN

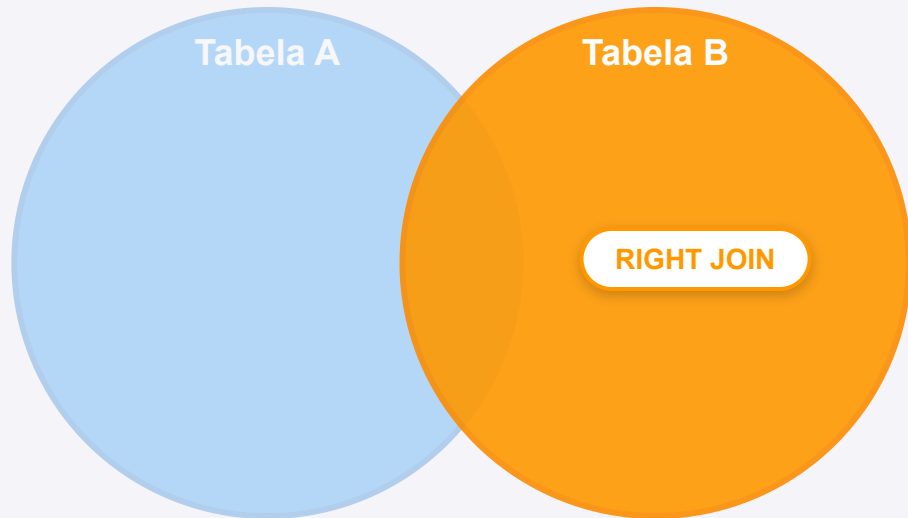


O RIGHT JOIN retorna **TODOS os registros da Tabela B (direita)**, independentemente de haver correspondência na Tabela A.

Quando há correspondência na Tabela A, os dados são incluídos. Quando não há, os campos da Tabela A aparecem como **NULL**.

```
SELECT *  
FROM  
    estudantes AS A  
RIGHT JOIN  
    notas AS B ON A.id = B.estudante_id;
```

RIGHT JOIN



Neste último exemplo vamos juntar todas as cláusulas aprendidas nesta aula para obter um resultado mais estruturado e que traga uma informação mais completa.

```
SELECT
    e.nome AS Estudante,
    d.nome AS Disciplina,
    n.nota AS Nota,
    CASE
        WHEN n.nota ≥ 7.0 THEN 'Aprovado'
        ELSE 'Reprovado'
    END AS Status_Aprovacao
FROM
    estudantes AS e
JOIN
    notas AS n ON e.id = n.estudante_id
JOIN
    disciplinas AS d ON n.disciplina_id = d.id
ORDER BY
    e.nome, d.nome;
```

Aqui usamos o JOIN para buscar informações de 3 tabelas e o CASE para criar uma nova coluna com um resultado consolidado que ajuda a entender a situação de cada aluno em cada disciplina.